Laporan Training dan Evaluasi

Model OCR Plat Nomor Indonesia

Proyek SmartPark — Indonesian License Plate Recognition

Berdasarkan pipeline training awal pada notebook
`kaggle/smartpark_ocr_training.ipynb`

Dokumen teknis internal
27 October 2026

1. Ringkasan Eksekutif

Dokumen ini merangkum seluruh proses pelatihan model OCR (Optical Character Recognition) plat nomor Indonesia untuk proyek SmartPark, dimulai dari pipeline awal pada notebook Kaggle `smartpark_ocr_training.ipynb`, dilanjutkan dengan berbagai eksperimen perbaikan, hingga evaluasi model pretrained. Lingkup pekerjaan dibatasi pada tahap **pengenalan teks** (recognition); tahap deteksi plat dengan YOLO dikerjakan terpisah dan tidak dibahas di sini.

Tujuan akhirnya adalah mendapatkan model yang mampu membaca teks plat dari gambar potongan (crop) plat secara otomatis dan akurat.

Pendekatan	Catatan	CER (val)	Exact Match
CRNN from-scratch	baseline awal	0.65	0/108
CRNN + ResNet18	transfer learning	0.4657	1/108
CRNN + TTA	decoding terbaik CRNN	0.4619	2/105
PARSeq + fine-tune	pretrained, tanpa augmentasi	0.065	64–66/105
PARSeq + augmentasi	**konfigurasi terbaik**	**0.029**	**85/105**

Kesimpulan utama. Hambatan utama ternyata bukan jumlah data, augmentasi, resolusi, atau regularisasi — melainkan **pilihan pendekatan model**. Mengganti CRNN yang dilatih dari nol dengan model pretrained **PARSeq** yang di-**fine-tune** menurunkan CER (Character Error Rate) dari sekitar 0.56 menjadi sekitar 0.065, dan menambahkan augmentasi realistis menurunkannya lagi ke **0.029**. Exact Match naik dari 0–1 plat menjadi **85 dari 105** plat (validation), dan **64 dari 108** pada test set. Perbaikan ini konsisten di kedua jenis domain data, sehingga kesenjangan antar-domain yang sebelumnya menjadi masalah kini hilang.

2. Langkah Pengerjaan

Pekerjaan dilakukan secara bertahap, dari menyiapkan data hingga memilih model akhir. Berikut urutan langkahnya:

1. **Audit data** — memeriksa 1.863 pasangan gambar–label: kecocokan gambar dengan label, label kosong, karakter di luar `A–Z`/`0–9`, dan duplikat.
2. **Pemisahan data** — menghapus gambar duplikat, lalu membagi train/val/test per-label sehingga tidak ada kebocoran antar-split.
3. **Validasi pipeline** — uji unit decoding CTC, smoke test **forward/backward**, dan uji overfit pada 24 sampel (hasil: 100% benar) untuk memastikan pipeline tidak bermasalah.
4. **Latih model awal** — CRNN (CNN + BiLSTM + CTC) dilatih **dari nol**.
5. **Transfer learning** — bagian CNN diganti dengan ResNet18 pretrained, diuji dua varian (`l2` dan `wp`).
6. **Perbaikan decoding** — membandingkan **greedy**, **beam search**, TTA, dan ensemble.
7. **Perluasan data** — memulihkan 355 crop plat dari anotasi deteksi, lalu retrain.
8. **Audit lanjutan & analisis kesalahan** — audit visual seluruh dataset dan analisis kesalahan per-domain.
9. **Eksperimen resolusi & fine-tuning** — menguji resolusi 48×240 / 64×320, **gradual unfreezing**, dan **weight decay** yang lebih kuat.
10. **Stabilisasi evaluasi** — memperbaiki determinisme dan replikasi 5 seed.
11. **Benchmark model pretrained** — menguji PARSeq, lalu melakukan **fine-tuning** pada data plat.

2.1. Akurasi Awal Model

Model pertama adalah CRNN yang dilatih **dari nol** (tanpa bobot pretrained). Inilah **akurasi awal** yang menjadi titik tolak seluruh perbaikan:

Metrik	Nilai	Arti
Train CER	0.0000	hafal sempurna (100% benar)
Test CER	0.6675	≈67% karakter salah
Test Exact Match	0/108	tidak ada plat benar penuh

Arti output: akurasi awal sangat rendah — CER ≈0.67 berarti sekitar dua per tiga karakter salah, dan tidak satu pun plat terbaca benar penuh. Model mampu **menghafal** data latih (train CER 0.0000, EM 831/831) tetapi gagal pada data baru. Angka inilah yang menjadi pembanding untuk setiap perbaikan berikutnya.

3. Pendahuluan

3.1. Tujuan

Membangun model OCR yang mampu mengubah gambar potongan plat nomor Indonesia menjadi teks alfanumerik yang benar, sebagai komponen dalam alur sistem parkir otomatis SmartPark:

Kamera → deteksi plat (YOLO) → **potong plat** → praproses → **OCR** → validasi plat

Tahap OCR adalah fokus laporan ini.

3.2. Metrik

- **CER (Character Error Rate)** — rasio kesalahan karakter (substitusi, hapus, sisip) terhadap total karakter. Semakin kecil semakin baik; 0 berarti sempurna.
- **Exact Match (EM)** — jumlah plat yang seluruh teksnya diprediksi benar persis.
- **Per-domain** — metrik dipisah menurut jenis sumber gambar (lihat Bagian 4).

Pemilihan konfigurasi dan checkpoint selalu menggunakan **validation set**. **Test set** tidak pernah dipakai untuk memilih model.

4. Dataset

Sumber data OCR: `plate_text_dataset`, dengan 1.863 pasangan gambar–label.

Integritas data (diverifikasi langsung, bukan sekadar dari nama file):

- 1.863 gambar dan 1.863 label — cocok 100%.
- 0 gambar tanpa label, 0 label kosong, 0 karakter di luar `A–Z` dan `0–9`.
- Semua gambar dapat dibuka.

Distribusi panjang label:

Panjang	Jumlah
4	2
5	14
6	55
7	1016
8	764
9	12

Jenis gambar. Berdasarkan isi gambar (bukan hanya nama file), data terbagi:

Jenis	Jumlah	Keterangan
`datatrain`	800	crop plat lebar, kualitas baik
`rf_detection`	347	crop persegi 416×416 berkonteks
foto utuh	716	foto kendaraan utuh (tidak dipakai langsung)

Dataset bersih. Setelah deduplikasi (gambar identik dihapus berdasarkan hash) dan penyaringan, diperoleh **1.044 crop plat** yang dibagi menjadi:

Split	Jumlah	Keterangan
train	831	data latih
val	105	pemilihan model
test	108	evaluasi akhir

Bebas kebocoran. Pemisahan data dilakukan per-label (satu nomor plat tidak muncul di lebih dari satu split). Hasil audit: 0 duplikat identik antar-split, 0 duplikat mirip antar-split, dan 0 kebocoran label. Ini penting agar evaluasi benar-benar mengukur kemampuan pada data baru.

5. Pipeline Training Awal (Notebook)

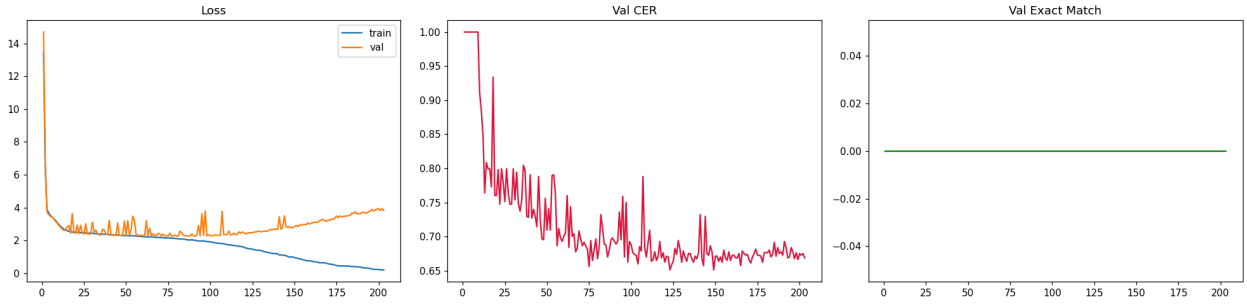
Notebook `kaggle/smartpark_ocr_training.ipynb` adalah pipeline awal yang menjadi acuan. Alur kerjanya, dari atas ke bawah:

1. **Konfigurasi & seed** — ukuran input 32×160 grayscale, batch 64, learning rate 3×10^{-4}, maksimum 80 epoch, early stopping (patience 12), seed 42.
2. **Audit dataset** — memeriksa jumlah baris, label kosong, karakter di luar charset, gambar hilang, dan duplikat.
3. **Visualisasi sampel** — montase gambar untuk verifikasi manual.
4. **Deduplikasi & split** — hapus duplikat, lalu bagi train/val/test per-label.
5. **Praproses** — resize menjaga aspek ke tinggi 32, **zero-pad** ke 32×160, lalu normalisasi; augmentasi ringan (rotasi, skala, kecerahan, blur) hanya saat training.
6. **Detektor plat (opsional)** — YOLOv8n dilatih dari dataset deteksi, dengan **fallback** OpenCV bila detektor gagal; dipakai untuk memotong foto kendaraan utuh.
7. **Model CRNN** — CNN sederhana + 2×BiLSTM(256) + linear, dilatih dengan CTC loss.
8. **Uji unit & smoke test** — memastikan decoding CTC dan alur **forward/backward** benar.
9. **Training** — AdamW + OneCycleLR, early stopping berdasarkan validation CER.
10. **Evaluasi akhir** — CER/EM pada test set, lalu menyimpan semua artefak + ZIP.

Model ini dilatih **dari nol** (tanpa bobot pretrained). Hasil lokal dari resep ini (varian yang diperpanjang, `artifacts/clean_run_fresh`):

Split	CER	Exact Match
val	0.6510	—
test	0.6675	0/108

Arti output: CER sekitar 0.67 berarti lebih dari dua per tiga karakter salah. Model dapat **menghafal** data latih dengan sempurna (train CER 0.0000, EM 831/831), tetapi gagal total pada data baru (EM 0). Ini menandakan masalah **generalisasi**, bukan bug pipeline atau label salah.



Gambar 1: Kurva training model CRNN from-scratch. Train loss terus turun, sedangkan validation CER berhenti membaik — tanda overfitting.

6. Arsitektur Model

6.1. CRNN + CTC (baseline)

Tahap	Keterangan
Input	grayscale 32×160
CNN	4 lapis konvolusi + MaxPool + BatchNorm
Sequence	BiLSTM 256, 2 layer, bidirectional
Output	Linear ⟶ 37 kelas (A–Z, 0–9, + blank CTC)
Loss	CTC (blank = 0)

6.2. CRNN + ResNet18 (transfer learning)

Versi perbaikan mengganti CNN from-scratch dengan **ResNet18 pretrained** (ImageNet). Dua varian diuji:

- **`wp` (width-preserving)** — lebar sekuens dijaga, sehingga model dapat mengenali lebih banyak karakter berturut-turut. Ini yang terbaik.
- **`l2`** — hanya memakai lapisan awal pretrained.

6.3. PARSeq (model pretrained)

PARSeq (**Permuted Autoregressive Sequence models**) adalah model transformer encoder–decoder untuk pengenalan teks (juara pada benchmark scene-text, ECCV 2022, lisensi Apache-2.0). Model ini:

- Memakai charset 94 karakter ASCII yang mencakup A–Z dan 0–9.
- Sudah dilatih pada data teks alami skala besar (bobot publik).
- Dilatih ulang (**fine-tune**) pada data plat kita dengan praproses aslinya (RGB, 32×128).

6.4. Dari-scratch vs Pretrained

Ada dua hal berbeda yang sering tertukar saat membahas model:

1. **Arsitektur** — bentuk/desain model (misalnya CRNN itu CNN + BiLSTM + CTC; PARSeq itu transformer).
2. **Titik awal bobot** — apakah model mulai dari angka **acak**, atau memakai bobot yang **sudah terlatih** sebelumnya.

Perbedaan terpenting ada di poin kedua. Model punya jutaan angka yang disebut **bobot** (isi "otak"-nya); bobot inilah yang membuat model bisa mengenali karakter.

- **Dilatih dari nol (from-scratch)** — arsitektur disusun sendiri, bobot mulai dari angka acak, dan model belajar semuanya hanya dari 831 gambar plat kita. Ibarat **bayi yang baru belajar membaca** dengan hanya 831 contoh: ia bisa menghafal contoh itu, tetapi belum memahami pola umumnya.
- **Pretrained** — memakai model yang **sudah dilatih orang lain** pada data raksasa (jutaan gambar/teks) sebelum kita pakai. Bobotnya sudah berisi "pengetahuan" mengenali bentuk karakter; kita tinggal **fine-tune** dengan data plat. Ibarat **orang dewasa yang sudah bisa membaca**, lalu cukup diajari gaya plat Indonesia.

Aspek	CRNN (rencana awal)	**PARSeq (pretrained)**
Pembuat arsitektur	kita sendiri	peneliti lain (publik)
Bobot mulai dari	acak	**sudah terlatih**
Yang kita lakukan	latih dari nol	**fine-tune** saja
Butuh data banyak?	ya	tidak terlalu
Hasil (CER)	0.56	**0.029**

Perlu dicatat: kita sempat memakai **campuran — CRNN + ResNet18 pretrained**. Di situ arsitekturnya tetap CRNN, tetapi bagian "penglihatan"-nya (ResNet18) memakai bobot pretrained. Hasilnya CER langsung turun dari 0.667 ⟶ 0.5236. Ini membuktikan bahwa **titik awal bobot pretrained** — bukan semata arsitektur — adalah faktor yang paling menolong.

7. Hasil Eksperimen CRNN

7.1. Transfer learning

Varian	val CER	test CER	test EM
`l2` (lr 1e-3)	0.5825	0.6203	0/108
`l2` (lr 3e-4)	0.6231	0.6762	0/108
`wp`	**0.4657**	**0.5236**	**1/108**

Arti output: backbone pretrained menurunkan test CER dari 0.6675 → 0.5236 (≈21% relatif). Ini menjadi **baseline terbaik** saat itu.

7.2. Perbaikan decoding

Mode decoding	test CER	test EM
greedy (acuan)	0.5236	1/108
beam search	0.6030	0/108
TTA + greedy	**0.5136**	1/108

Arti output: beam search justru lebih buruk karena menambah karakter berlebih (insertion) pada output yang pendek. **TTA + greedy** memberi perbaikan kecil tanpa biaya tambahan.

7.3. Perluasan data (recovery)

Sebanyak 355 crop plat baru berhasil dipulihkan dari anotasi deteksi dan digabungkan (train 831 → 1186). Hasilnya:

Metrik	Baseline	Setelah perluasan
val CER	**0.4657**	0.5470
test CER	**0.5236**	0.6005
test EM	**1/108**	0/108

Arti output: menambah data justru **memperburuk** hasil secara keseluruhan. Analisis per-domain menunjukkan data baru berasal dari **domain berbeda** (foto motor miring) dibanding data lama (plat mobil terang). Model kehabisan kapasitas menyeimbangkan dua domain — fenomena **domain shift**, bukan label salah.



Gambar 2: Contoh crop plat yang berhasil dipulihkan. Label terverifikasi cocok secara visual, namun gaya gambarnya berbeda dari data lama.

8. Audit Kualitas dan Analisis Kesalahan

8.1. Audit visual seluruh dataset

Seluruh 1.863 gambar diperiksa langsung (bukan hanya dari nama file). Hasil: **0 label salah**, 0 file hilang, 0 gambar rusak. Kategori kualitas:

Kategori	Jumlah
crop layak OCR	987
foto kendaraan utuh	716
gelap / blur / ekstrem	75
crop terlalu kecil	69
duplikat mirip	16

Arti output: tidak ada label noise sistematis, sehingga kualitas data bukan penyebab utama kegagalan — masalahnya ada di model.

8.2. Analisis kesalahan OCR

Komposisi kesalahan pada baseline (val + test):

Substitusi	Hapus	Sisip	Total
661	17	100	778

- Panjang prediksi hampir selalu benar → model gagal **membedakan karakter**, bukan menghitung jumlah karakter.
- Pasangan karakter yang sering tertukar: `8↔0`, `9↔1`, `6↔5`, `4↔3`, `T↔S` — semuanya bentuk mirip.
- Performa per-domain: `datatrain` ≈ 0.41, `rf_detection` ≈ 0.68.
- Temuan kunci: crop `rf_detection` (416×416 persegi) setelah praproses hanya menyisakan **≈4,6 piksel per karakter** (vs `datatrain` ≈13,7).

Arti output: model kehilangan detail karakter, terutama pada plat berkonteks yang menyusut. Ada kesenjangan jelas antar-domain.



Gambar 3: Atas: gambar asli. Bawah: gambar setelah praproses (yang benar-benar masuk ke model). Pada domain `rf_detection`, plat menjadi sangat kecil dan sulit dibaca.

9. Eksperimen Resolusi, Fine-tuning, dan Regularisasi

9.1. Eksperimen resolusi input

Hipotesis: menaikkan resolusi input akan mengembalikan detail karakter.

Resolusi	val CER	`datatrain`	`rf_detection`
32×160 (baseline)	**0.4657**	**0.3876**	0.6637
48×240	0.5343	0.5310	**0.5426**
64×320	0.6599	0.6389	0.7130

Arti output: resolusi lebih tinggi **memperbaiki** domain sulit (`rf_detection` 0.66 ⟶ 0.54) tetapi **memperburuk** domain utama (`datatrain` 0.39 ⟶ 0.53), sehingga keseluruhan lebih buruk. Pemeriksaan visual menunjukkan gambar pada 48×240 jauh lebih terbaca, namun prediksi tetap salah ⟶ **resolusi bukan akar masalah**. Baseline 32×160 dipertahankan.

9.2. Fine-tuning bertahap dan regularisasi

- **Eksperimen A** (gradual unfreezing): val CER **0.5914**.
- **Eksperimen B** (weight decay lebih kuat): val CER **0.6003**.
- Pembanding: tiga run baseline segar memberi 0.5609 / 0.5952 / 0.6193.

Arti output: kedua strategi **tidak meningkatkan** generalisasi; hasilnya berada di dalam rentang variasi antar-run. Temuan penting: baseline historis 0.4657 ternyata **tidak dapat direproduksi** secara konsisten.

10. Stabilisasi Evaluasi

Ditemukan bahwa hasil training bervariasi antar-run (operasi GPU/LSTM non-deterministik). Ditambahkan mode deterministik (opsional), lalu baseline dijalankan ulang dengan **5 seed**:

Statistik	val CER
mean	0.5579
std	0.0546
min	0.4530
max	0.6117

Arti output: performa wajar baseline ≈ **0.56 ± 0.05**, bukan 0.4657 (angka itu adalah ujung terbaik dari sebaran). Setiap perbandingan selanjutnya harus menyebut rentang ini agar tidak menyesatkan. Evaluasi kini dapat diulang dan dipercaya.

11. Benchmark Model Pretrained: PARSeq

11.1. Uji awal (zero-shot)

PARSeq **tanpa fine-tuning** diuji langsung pada data plat:

Kondisi	val CER	val EM
PARSeq zero-shot	1.0406	0/105

Arti output: model pretrained generik **tidak mengenali** font/format plat Indonesia. **Fine-tuning wajib dilakukan.**

11.2. Setelah fine-tuning

Fine-tuning hanya memakai train set; checkpoint dipilih dari validation set.

Model	val CER	val EM	`datatrain`	`rf_detection`
CRNN baseline (5 seed)	0.5579 ± 0.055	0.2	—	—
CRNN baseline (terbaik)	0.4530	1/105	0.3699	0.6637
PARSeq tanpa augmentasi	0.0647	66/105	0.0726	0.0448
PARSeq + aug (5 seed)	0.0370 ± 0.0048	78,6/105	—	—
PARSeq + aug (terbaik)	**0.0292**	**85/105**	0.0354	0.0135

Arti output:

- Validation CER turun drastis (≈0.56 ⟶ ≈0.065 untuk fine-tune biasa, lalu **≈0.029** setelah augmentasi). Exact Match naik dari 0–1 menjadi **85 dari 105** plat pada validation.
- **Kesenjangan antar-domain hilang**: `rf_detection` turun 0.66 ⟶ **0.013** dan `datatrain` 0.39 ⟶ 0.035. PARSeq kini justru paling kuat pada domain yang tadinya paling sulit.
- Hasil **konsisten di 5 seed** (CER 0.029–0.043, std 0.005) ⟶ peningkatan bukan kebetulan satu run.
- Biaya: model 1,66× lebih besar dan lebih lambat (5–20 ms/gambar), masih memadai untuk kebutuhan parkir.
- Sisa kesalahan berupa **near-miss** satu karakter, bukan kegagalan sistematis.

11.3. Augmentasi realistis

Augmentasi (rotasi ±3°, skala, geser, kecerahan/kontras, blur, noise) diterapkan hanya saat training. Ini tuas terbesar kedua setelah memakai model pretrained:

Kondisi	val CER	val EM
PARSeq tanpa augmentasi	0.0647	66/105
PARSeq + augmentasi	**0.0292**	**85/105**

Arti output: augmentasi menurunkan CER lebih dari separuh (0.065 ⟶ 0.029) dan menaikkan Exact Match dari 66 menjadi 85. Model tidak lagi sekadar menghafal tampilan data latih.

11.4. Evaluasi test set (sekali, final)

Setelah konfigurasi dipilih dari validation, model terbaik (seed 44) dievaluasi **satu kali** pada test set 108 plat yang belum pernah dilihat:

Split	CER	Exact Match
validation (105)	0.0292	85/105
test (108)	**0.0844**	**64/108**

Arti output: pada data yang benar-benar baru, CER ≈0.084 dengan 64/108 plat terbaca sempurna. Test lebih sulit dari validation (ada plat blur/gelap), tetapi tetap **jauh** lebih baik daripada CRNN (test CER 0.52, EM 1/108).



Gambar 4: Contoh kesalahan PARSeq yang tersisa. Sebagian besar hanya berbeda satu karakter pada plat yang blur/gelap.

12. Perbandingan CRNN vs PARSeq

Bagian ini membandingkan kedua pendekatan secara langsung pada **validation set** yang sama (105 plat), agar terlihat jelas di mana masing-masing unggul.

12.1. Perbandingan kuantitatif

Aspek	CRNN + ResNet18	**PARSeq + augmentasi**
Jenis model	CRNN (CNN + BiLSTM + CTC)	transformer encoder–decoder
Dibangun dari awal?	tidak — backbone pretrained ImageNet	tidak — pretrained scene-text
Jumlah parameter	14,3 juta	23,8 juta
Metode decoding	greedy CTC	autoregresif
val CER	**0.4530**	**0.0292**
val Exact Match	**1/105**	**85/105**
`datatrain` CER	0.3699	**0.0354**
`rf_detection` CER	0.6637	**0.0135**
Latency (ms/gambar)	**0,8–2,9**	4,8–20
Ukuran relatif	**1× (lebih ringan)**	1,66×

12.2. Perbandingan kualitatif

Aspek	CRNN + ResNet18	**PARSeq + augmentasi**
Akurasi	rendah (CER ≈0.45)	**jauh lebih tinggi (CER ≈0.03)**
Exact Match	hampir nol (0–1 plat)	**tinggi (85 dari 105 plat)**
Konsistensi domain	jomplang (`datatrain` ≫ `rf_detection`)	**merata, bahkan terbaik di `rf_detection`**
Test set	CER 0.52, EM 1/108	**CER 0.084, EM 64/108**
Kesalahan	substitusi digit mirip	near-miss pada plat blur
Kecepatan & ukuran	**lebih cepat & ringan**	sedikit lebih berat
Kesiapan produksi	kurang (akurasi belum memadai)	**siap sebagai kandidat utama**

Arti output:

- **Akurasi:** PARSeq unggul sangat jauh — CER sekitar **15× lebih rendah** dan Exact Match dari 1 menjadi 85 plat.
- **Pemerataan domain:** keunggulan PARSeq bukan hanya menang di satu domain. Domain yang tadinya paling sulit bagi CRNN (`rf_detection`, CER 0.66) justru menjadi yang terbaik bagi PARSeq (CER 0.013) — kesenjangan antar-domain hilang.
- **Tukar tambah:** CRNN lebih ringan dan cepat (0,8–2,9 ms vs 4,8–6,1 ms per gambar), tetapi selisih ini tidak berarti untuk kebutuhan parkir. PARSeq 1,66× lebih besar namun masih nyaman dijalankan di GPU yang tersedia.
- **Catatan keadilan:** perbandingan ini membandingkan **pendekatan**, bukan satu-faktor. PARSeq juga memanfaatkan bobot pretrained yang sudah dilatih pada data teks skala besar — dan justru inilah sumber keunggulannya.

Ringkas: untuk akurasi, **PARSeq jelas menang** dan layak dijadikan model utama. CRNN tetap berguna sebagai cadangan ringan bila sumber daya sangat terbatas.

13. Kesimpulan dan Rekomendasi

13.1. Pelajaran kunci

1. **Pendekatan pretrained mengalahkan training from-scratch.** Fine-tuning model transformer pretrained memberi lompatan terbesar (CER 0.56 → 0.065).
2. **Augmentasi realistis menurunkan CER lebih dari separuh** lagi (0.065 → 0.029).
3. **Menambah data tanpa konsistensi domain bisa merugikan** (domain shift).
4. **Resolusi, regularisasi, dan fine-tuning bertahap tidak menyelesaikan masalah utama** — hanya memberi perubahan kecil atau bahkan memperburuk.
5. **Beam search dan TTA tidak menolong** pada model ini (greedy sudah terbaik).
6. **Evaluasi harus multi-seed.** Hasil 5 seed: CER 0.029–0.043 (std 0.005).
7. **Verifikasi integritas data penting** dan terbukti bersih.

13.2. Rekomendasi

1. **Adopsi PARSeq + augmentasi** sebagai model OCR utama untuk SmartPark (dasar: val CER 0.029 vs ≈0.56, val EM 85/105; test CER 0.084, EM 64/108).
2. **Simpan CRNN sebagai fallback ringan** — jangan dihapus.
3. **Evaluasi pada data kamera nyata** SmartPark sebagai uji independen berikutnya (test 108 sudah dipakai sekali).
4. **Uji efisiensi deployment** (ONNX/TensorRT, latency CPU) karena PARSeq lebih besar dan lebih lambat.
5. **Post-processing validasi format plat Indonesia** (awalan wilayah + panjang) untuk menaikkan Exact Match.
6. **Belum dilakukan (di luar lingkup):** melatih detektor YOLO dan memotong seluruh foto kendaraan utuh.

13.3. Keterbatasan

- Angka pemilihan model berasal dari **validation set**; test set dipakai **sekali** pada model terpilih (seed 44).
- Perbandingan CRNN vs PARSeq mencakup perbedaan praproses dan arsitektur (merupakan bagian dari pendekatan), bukan perbandingan satu-faktor.
- Latency diukur pada satu GPU (RTX 2050), bukan kondisi CPU/deployment.
- Seed 45 terhenti di epoch 62; checkpoint terbaiknya tetap dipakai (ep 35).

Angka pada laporan ini diambil langsung dari berkas hasil eksperimen di `reports/improvement_v1/` dan `artifacts/`. Laporan rinci per-tahap tersedia di `audit_summary.md`, `resolution_report.md`, `phaseF_finetune_report.md`, dan `ocr_benchmark_report.md`.